

Lecture 06 — The while loop, in practice — one step too many, one too few

Algorithmic Thinking · Week 3, Session 6 — Friday 9 October 2026, 17:35–18:55
(after Lab 3 and the final diagnostic, 16:30–17:30)

Leopoldo Rodríguez Salazar · Tecnológico de Monterrey

Contents

Overview	1
Opener — The shortest job first (10 min)	1
Notes	2
Stretch — if the session was easy	5
After the session	5

Objective. Write four loops inside functions, each tested from the shell at its boundary, and name the off-by-one before running.

Overview

Tuesday gave you the loop. Today you write four of them, and every one is built around the same trap: **the loop that runs once too often, or once too few**. 6, not 5. 55 becoming 65. `digits(0)`. The spec says *down to 1* and the program prints 0.

Off-by-one is not a beginner’s bug. It is *the* bug — the one professionals still write. The cure is not care; it is a habit: **before you run the loop, write down what the first turn and the last turn do**. Today’s exercises are chosen so that the boundary is always where the marks are, and the shell is where you check it in four seconds.

Everything is a function with a main. That is no longer a topic; it is the format.

Opener — The shortest job first (10 min)

Algorithms to Live By, chapter 5, *Scheduling*. You have five things to do tonight and one evening. Which order?

Put them on the slide with their durations: *reply to an email (5 min)*, *HW4 (90 min)*, *buy groceries (30 min)*, *call home (15 min)*, *read for Tuesday (40 min)*. Ask the room for an order and a reason. Someone will say “the urgent one first”; someone will say “the big one first, get it out of the way”.

Then the result from 1954 (W. E. Smith): if the goal is that **the sum of everyone’s waiting time is as small as possible**, there is exactly one right order — **Shortest Processing Time first**. Do the 5, then the 15, then the 30, then the 40, then the 90. Every other order makes the total wait larger. The intuition: a long task at the front

makes *everything behind it* wait; a short task at the front makes everything wait only a little.

Compute it with them: SPT order gives completion times 5, 20, 50, 90, 180 — total **345**. Longest first: 90, 130, 160, 175, 180 — total **735**. Same five tasks, same evening, twice the waiting.

Now look at how you *did* it: **while there are tasks left: pick the shortest remaining one, do it, cross it off**. A loop. With a quantity that moves toward the exit — the list gets shorter every turn. (The book's caveat: if the tasks have *deadlines*, the rule changes to Earliest Due Date first. Different goal, different algorithm — the goal comes first.)

Your turn, 3 minutes, on paper: write SPT as an algorithm. INPUT: a list of tasks with durations. OUTPUT: the order. The loop, the exit, and the one step that chooses. Keep it: in week 7 you will write it in Python, with a list.

Notes

The smell, revisited — what broke in the lab

Two answers from the room, no more. The ones you want: “it never stopped” (missing step, or step outside the block) and “it asked one time too many” (the test). Both are today.

Exercise 1 — countdown(n) (*the test*)

Open 01_countdown.py. Spec: *display n, n-1, ..., 1, one per line, then Liftoff!*

```
def countdown(n):
    while n > 0:
        print(n)
        n = n - 1
    print("Liftoff!")
```

Shell: `countdown(3)` → 3 2 1 Liftoff!. `countdown(1)` → 1 Liftoff!. **`countdown(0)`** → just Liftoff! — the block ran zero times, and that is what the spec implies. Change the test to `n >= 0` and 0 appears; the spec said *down to 1*. One character, one extra turn.

Notice we are changing the **parameter** `n` inside the function. That is allowed — it is the function's own copy (A4 on Tuesday's exam). The caller's variable is untouched.

Exercise 2 — digits(n) (*the start*)

Open 02_digits.py. Spec: *return how many digits a non-negative whole number has*.

```
def digits(n):
    count = 1
    while n >= 10:
        n = n // 10
        count = count + 1
    return count
```

The step is $n // 10$ — integer division drops the last digit: $1234 // 10$ is 123. Shell: $\text{digits}(1000) \rightarrow 4$, $\text{digits}(999) \rightarrow 3$, $\text{digits}(10) \rightarrow 2$, $\text{digits}(7) \rightarrow 1$.

Now the start. The obvious version starts $\text{count} = 0$ with $\text{while } n > 0$. It gives $\text{digits}(0) \rightarrow 0$. Does zero have no digits? The spec says *non-negative*, so 0 is a legal input and the answer must be 1. Starting count at 1 and testing $n >= 10$ fixes it — every number has at least one digit, and the loop counts the *extra* ones. **Where the counter starts is a decision about the smallest input.**

Exercise 3 — `play(secret)` (*the step order, and random*)

Open `03_guess.py`. The computer picks a number 1-100; the user guesses; the program says *Higher* or *Lower*; at the end, the number of attempts.

```
import random

def play(secret):
    attempts = 0
    guess = int(input("Guess: "))
    attempts = attempts + 1
    while guess != secret:
        if guess < secret:
            print("Higher.")
        else:
            print("Lower.")
        guess = int(input("Guess: "))
        attempts = attempts + 1
    return attempts

def main():
    secret = random.randint(1, 100)
    print("Found it in", play(secret), "attempts.")
```

Three things. The **sentinel idiom** from Tuesday, with the “sentinel” now being *the right answer*: first `input()` before the loop, the next one last inside it. The **counter** goes up right after each `input()` — put it anywhere else and you are off by one (try: move `attempts = attempts + 1` above the first `input()`... and the count still works; move it *out* of the loop and it never passes 1). And `random.randint(1, 100)` — includes both ends. The `random` module is imported at the top; this is the self-study page’s first appearance in anger.

Shell test with a *known* secret — that is why `play` takes `secret` as a parameter instead of picking it: `play(50)`, type 50 $\rightarrow$ 1. `play(50)`, type 25, 75, 50 $\rightarrow$ 3. A function you can test is a function whose randomness lives in `main`.

Stretch 01 asked for an algorithm that always finds the number in seven questions. Play it that way: 50, then 25 or 75... How many did you need?

Exercise 4 — `gcd(a, b)` (*termination*)

Open `04_gcd.py`. Euclid, 300 BC — from the stretch problems of lecture 01, now as code:

```
def gcd(a, b):
    while a != b:
        if a > b:
            a = a - b
        else:
            b = b - a
    return a
```

$\text{gcd}(48, 18) \rightarrow 6$: $48-18 = 30$, $30-18 = 12$, $18-12 = 6$, $12-6 = 6$, equal — stop.
 $\text{gcd}(1071, 462) \rightarrow 21$. $\text{gcd}(7, 7) \rightarrow 7$ without entering the loop.

Why does it stop? **Every turn makes one of the two numbers strictly smaller, and neither can go below 1** (they are positive, and we always subtract the smaller from the larger). So the sum $a + b$ goes down every turn and cannot go below 2. Name the quantity, name the floor — that is a termination proof, and it is the only kind of argument that ever convinces anyone a loop ends.

Now break the promise: $\text{gcd}(5, 0)$. $a > b$, $a = 5 - 0 = 5$, forever. Ctrl+C. The docstring said *positive integers*; the caller lied. Whose bug is it? **The caller's** — and yet in main *you* are the caller, so main must check before it calls. That is the division of labour we have had since week 2: functions compute, main reads, checks and prints.

The gallery

Open 05_off_by_one_gallery.py. Four loops, each exactly one step from right. Predict all four on paper before running: (a) prints 1-4, the spec said 1-5. (b) prints b four times for $n = 3$. (c) gives 14 for $n = 4$ — the sum should be 10. (d) counts the -1 and misses the first reading. For each, say which of the three places the bug is in: **test, start, or step order**.

In class — the four, as files (*individual, from 18:05; whatever is not finished becomes homework for your own benefit, not for Canvas*)

Each exercise is **one function and a main**, in the exam format, algorithm on paper first. Shell tests written under the algorithm before you run `main()`.

File	Function	Shell tests you must write down
A_Matricula_w1.py	countdown(n)	countdown(3), countdown(1), countdown(0)
A_Matricula_w2.py	digits(n) $\rightarrow$ number of digits	digits(0) $\rightarrow 1 \cdot$ digits(7) $\rightarrow 1 \cdot$ digits(10) $\rightarrow 2 \cdot$ digits(1000) $\rightarrow 4$
A_Matricula_w3.py	play(secret) $\rightarrow$ attempts; main picks the secret with <code>random.randint(1, 100)</code>	play(50) typing 50 $\rightarrow 1$; typing 25 75 50 $\rightarrow 3$

File	Function	Shell tests you must write down
A_Matricula_w4.py	gcd(a, b) by subtraction; main refuses non-positive input with Both numbers must be positive.	gcd(48, 18) → 6 · gcd(1071, 462) → 21 · gcd(7, 7) → 7; main with 5 0 → the message

Photograph the algorithms and upload them with the four .py files to the **Week 3 evidence box** (not graded, but it is your record, and I read it). Finished all four? Make play tell the user how far from seven they were.

Stretch — if the session was easy

Optional, ungraded, harder than they look.

1. **Euclid, faster.** Replace subtraction with `%`: `while b != 0: a, b = b, a % b`. Trace gcd(1071, 462) both ways and count the turns. Then argue why the `%` version also stops — the quantity is different. Which version would you trust with gcd(10**9, 1)?
2. **The binary search guesser.** Write `robot_play(secret)` that plays play's game *without a human*: it keeps low and high, always guesses the middle, and counts attempts. Run it for every secret from 1 to 100 (a loop around a loop) and report the maximum attempts. Is it 7? Then make the range 1-1000 and predict the maximum before running.
3. **Reverse, not count.** `reverse_digits(n)` returns the number with its digits reversed — 1234 → 4321, 1200 → 21. Same loop as `digits`, with `% 10` to pull the last digit and an accumulator built with `* 10 +`. Then the question: for which inputs does `reverse_digits(reverse_digits(n)) != n`, and is that a bug or the spec?

After the session

Homework 4 — assignments/hw-04-while.md. Due Sunday 11 October, 23:59, on Canvas. One file, A_Matricula_hw4.py: `collatz_steps(n)` and `reverse_digits(n)` plus a main menu, shell tests written down, algorithm on paper. Both loops need a termination sentence.

Also before Tuesday:

1. **Lab 3 files** — C1_matricula.py, C2_matricula.py + the photo of your algorithms → Canvas → *Laboratory 3*, **Sunday 11 October, 23:59**.
2. **Quick exam 3 — Tuesday 13 October, first 20 minutes:** the while loop. Read / predict / fix on today's gallery, and one loop to write as a function. Rubric on the site.
3. **Self-study: counter-controlled repetition** (Canvas, Week 4) — ten minutes, so for on Tuesday is a word you have seen.
4. The levelling module is closed. If you did not finish, the final diagnostic today was your last word on it; the course moves on.